

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3.8

UPGRADING TO POSTGIS SUPPORT

 Copy as Markdown

Upgrading to PostGIS support

New Serverpod projects already include PostGIS, both in the embedded PostgreSQL and in the Docker image. This guide is for older Docker setups and external PostgreSQL servers. To use geography fields in your models there, you need a PostgreSQL instance with the PostGIS extension installed.

INFO

This upgrade is only necessary if you want to use geography fields in your models. If you do not plan to use geography fields, you can skip this upgrade.

WARNING

If trying to use geography fields without upgrading, you will encounter an error when applying migrations.

For Docker-based environments

1. Update your `docker-compose.yml` to use `ghcr.io/serverpod/postgres:16`, the Serverpod PostgreSQL image. It ships with [PostGIS](#) and [pgvector](#) already installed:

```
services:
  postgres:
    image: ghcr.io/serverpod/postgres:16 # ← Change from postgres image here
    ports:
      - '8090:5432'
    environment:
      POSTGRES_USER: postgres
      POSTGRES_DB: <projectname>
      POSTGRES_PASSWORD: <DB_PASSWORD>
    volumes:
      - <projectname>_data:/var/lib/postgresql/data
```



```
# Other services...

postgres_test:
  image: ghcr.io/serverpod/postgres:16 # ← Change from postgres image here
  ports:
    - '9090:5432'
  environment:
    POSTGRES_USER: postgres
    POSTGRES_DB: <projectname>_test
    POSTGRES_PASSWORD: <DB_TEST_PASSWORD>
  volumes:
    - <projectname>_test_data:/var/lib/postgresql/data
```

If your `docker-compose.yaml` came from Serverpod 4.0.0-beta.2 or later, it already references this image and needs no change.

2. Recreate your containers to use the new image:

```
docker compose down
docker compose up -d
```

3. Create your first geography field in a model:

```
class: Store
table: store
fields:
  name: String
  location: GeographyPoint
```

4. Generate and apply a migration:

```
$ serverpod create-migration
$ dart run bin/main.dart --apply-migrations
```

For more details on creating and applying migrations, see the [Migrations](#) section.

The PostGIS extension will be automatically enabled during the first migration that includes a geography column.

For managed PostgreSQL services

For cloud providers (AWS RDS, Google Cloud SQL, Azure Database, etc.), ensure that the PostGIS extension is available on your PostgreSQL instance. Most major managed services support PostGIS with no additional setup required. If available, the extension will be enabled automatically when applying the migration.

If the cloud provider instructs you to run a `CREATE EXTENSION postgis;` command, you can skip that step. Serverpod handles it automatically during migration.

Troubleshooting

If you encounter issues with PostGIS:

- Verify that your PostgreSQL version is 12 or later.
- Check that the PostGIS extension is properly installed on the instance.
- Ensure your database user has the necessary permissions to create extensions.